



ESTRUTURAS DE DADOS

EXERCÍCIOS DE IMPLEMENTAÇÃO, CONCEITO E ALGORITMOS

LISTA DE EXERCÍCIOS

BALANÇO TÉCNICO

TIPOS ABSTRATOS DE DADOS

- ① Implemente um TAD **Ponto** que represente um ponto no plano cartesiano com coordenadas (x, y).

O TAD deve fornecer as seguintes operações:

- Criar um ponto com coordenadas x e y
- Obter as coordenadas x e y
- Calcular a distância do ponto até a origem (0, 0)
- Calcular a distância entre dois pontos
- Exibir o ponto no formato “(x, y)”

Exemplo de uso

```
Ponto p1 = criarPonto(3, 4);  
Ponto p2 = criarPonto(0, 0);
```

```
Distância até origem: 5.00  
Distância entre p1 e p2: 5.00  
Coordenadas: (3, 4)
```

- ② Implemente um TAD **Data** que represente uma data com dia, mês e ano.

O TAD deve fornecer as seguintes operações:

- Criar uma data (com validação de data válida)
- Verificar se o ano é bissexto
- Comparar duas datas (anterior, igual ou posterior)
- Calcular a diferença em dias entre duas datas
- Adicionar N dias a uma data
- Exibir a data no formato “DD/MM/AAAA”

Exemplo de uso

```
Data d1 = criarData(15, 3, 2024);  
Data d2 = criarData(20, 3, 2024);
```

```
Data válida: true  
Bissexto: true  
Comparação: d1 é anterior a d2  
Diferença: 5 dias  
d1 + 10 dias: 25/03/2024
```

- 3 Implemente um TAD Conjunto que represente um conjunto matemático de inteiros.

O TAD deve fornecer as seguintes operações:

- Criar conjunto vazio
- Inserir elemento (sem repetições)
- Remover elemento
- Verificar se elemento pertence ao conjunto
- Obter tamanho do conjunto
- União de dois conjuntos
- Interseção de dois conjuntos
- Diferença entre dois conjuntos
- Verificar se um conjunto é subconjunto de outro
- Exibir elementos do conjunto

Exemplo de uso

Conjunto A = {1, 2, 3, 4}

Conjunto B = {3, 4, 5, 6}

$A \cup B = \{1, 2, 3, 4, 5, 6\}$

$A \cap B = \{3, 4\}$

$A - B = \{1, 2\}$

$3 \in A$: true

$A \subseteq B$: false

- 4 Implemente um TAD **MatrizEsparsa** que represente uma matriz esparsa (com muitos elementos zero) de forma eficiente.

O TAD deve armazenar apenas elementos não-nulos usando lista encadeada e fornecer:

- Criar matriz esparsa com dimensões $M \times N$
- Inserir elemento em posição (i, j)
- Acessar elemento em posição (i, j)
- Somar duas matrizes esparsas
- Multiplicar duas matrizes esparsas
- Transpor a matriz
- Exibir matriz completa (incluindo zeros)

Exemplo de uso

```
MatrizEsparsa M1(5, 5);  
M1.inserir(0, 0, 10);  
M1.inserir(2, 3, 20);  
M1.inserir(4, 4, 30);
```

Elementos armazenados: 3 (de 25 possíveis)

$M1[2][3] = 20$

$M1[1][1] = 0$

Matriz transposta:

```
10  0  0  0  0  
 0  0  0  0  0  
 0  0  0 20  0  
 0  0  0  0  0  
 0  0  0  0 30
```

LISTAS E ARRAYS

- 5 Crie um programa em C++ que implemente uma lista encadeada simples (singly linked list) e exiba seus elementos. O programa deve permitir ao usuário inserir valores na lista e, ao final, exibir todos os elementos na ordem em que foram inseridos.

Exemplo de entrada

```
Digite a quantidade de elementos: 6
Digite o elemento 1: 11
Digite o elemento 2: 9
Digite o elemento 3: 7
Digite o elemento 4: 5
Digite o elemento 5: 3
Digite o elemento 6: 1
```

Exemplo de saída

```
Lista encadeada: 11 9 7 5 3 1
```

- 6 Crie um programa em C++ que crie uma lista encadeada simples de n nós e exiba seus elementos em ordem reversa. O programa deve solicitar ao usuário a quantidade de elementos e seus valores, e depois exibir a lista do último elemento para o primeiro.

Exemplo de entrada

```
Digite a quantidade de elementos: 6
Digite o elemento 1: 11
Digite o elemento 2: 9
Digite o elemento 3: 7
Digite o elemento 4: 5
Digite o elemento 5: 3
Digite o elemento 6: 1
```

Exemplo de saída

```
Lista original: 11 9 7 5 3 1
Lista reversa: 1 3 5 7 9 11
```

- 7 Crie um programa em C++ que crie uma lista encadeada simples de n nós e conte o número total de nós. O programa deve permitir inserir elementos na lista e, ao final, exibir a quantidade total de nós presentes.

Exemplo de entrada

```
Digite a quantidade de elementos: 7
Digite o elemento 1: 13
Digite o elemento 2: 11
Digite o elemento 3: 9
Digite o elemento 4: 7
Digite o elemento 5: 5
Digite o elemento 6: 3
Digite o elemento 7: 1
```

Exemplo de saída

```
Lista encadeada: 13 11 9 7 5 3 1
Número de nós na lista: 7
```

- 8 Crie um programa em C++ que insira um novo nó no início de uma lista encadeada simples. O programa deve criar uma lista com alguns elementos, solicitar um novo valor ao usuário e inseri-lo no início da lista.

Exemplo de entrada

```
Lista original: 13 11 9 7 5 3 1
Digite o valor a ser inserido no início: 0
```

Exemplo de saída

```
Lista após inserção no início: 0 13 11 9 7 5 3 1
```

- 9 Crie um programa em C++ que insira um novo nó no final de uma lista encadeada simples. O programa deve criar uma lista com alguns elementos, solicitar um novo valor ao usuário e inseri-lo no final da lista.

Exemplo de entrada

Lista original: 13 11 9 7 5 3 1
Digite o valor a ser inserido no final: 0

Exemplo de saída

Lista após inserção no final: 13 11 9 7 5 3 1 0

- 10 Crie um programa em C++ que encontre o elemento do meio de uma lista encadeada. O programa deve criar uma lista e determinar qual elemento está na posição central. Se a lista tiver número par de elementos, retorne o segundo elemento do meio.

Pseudo-código

```
slow = head
fast = head
enquanto fast ≠ null e fast.next ≠ null:
    slow = slow.next
    fast = fast.next.next
retornar slow.valor
```

Exemplo de entrada 1

Lista: 7 5 3 1

Exemplo de saída 1

Elemento do meio: 3

Exemplo de entrada 2

Lista: 9 7 5 3 1

Exemplo de saída 2

Elemento do meio: 5

- ⑪ Crie um programa em C++ que insira um novo nó no meio de uma lista encadeada simples. O programa deve criar uma lista, calcular a posição do meio e inserir o novo valor nessa posição.

Exemplo de entrada

Lista original: 7 5 3 1

Digite o valor a ser inserido no meio: 9

Exemplo de saída (após primeira inserção)

Lista após inserção: 7 5 9 3 1

Exemplo de saída (após segunda inserção)

Lista após inserção: 7 5 9 11 3 1

- ⑫ Crie um programa em C++ que obtenha o n-ésimo nó de uma lista encadeada simples. O programa deve criar uma lista, solicitar ao usuário uma posição e retornar o valor do nó nessa posição (1-indexed).

Exemplo de entrada

Lista: 7 5 3 1

Digite a posição desejada: 3

Exemplo de saída

Valor na posição 3: 3

- 13 Crie um programa em C++ que insira um novo nó em qualquer posição de uma lista encadeada simples. O programa deve criar uma lista, solicitar ao usuário a posição e o valor a ser inserido, e inserir o nó na posição especificada (1-indexed).

Exemplo de entrada 1

Lista original: 7 5 3 1
Digite a posição: 1
Digite o valor: 12

Exemplo de saída 1

Lista atualizada: 12 7 5 3 1

Exemplo de entrada 2

Lista atual: 12 7 5 3 1
Digite a posição: 4
Digite o valor: 14

Exemplo de saída 2

Lista atualizada: 12 7 5 14 3 1

- 14 Crie um programa em C++ que delete o primeiro nó de uma lista encadeada simples. O programa deve criar uma lista, exibi-la, remover o primeiro elemento e exibir a lista atualizada.

Exemplo de entrada

Lista original: 13 11 9 7 5 3 1

Exemplo de saída

Lista após remoção do primeiro nó: 11 9 7 5 3 1

- 15 Crie um programa em C++ que delete um nó do meio de uma lista encadeada simples. O programa deve criar uma lista, calcular a posição do meio, remover o nó dessa posição e exibir a lista atualizada. O processo pode ser repetido até que reste apenas um elemento.

Exemplo de entrada

Lista original: 9 7 5 3 1

Exemplo de saída (após primeira remoção)

Lista após remover o meio: 9 7 3 1

Exemplo de saída (após segunda remoção)

Lista após remover o meio: 9 7 1

Exemplo de saída (após terceira remoção)

Lista após remover o meio: 9 1

Exemplo de saída (após quarta remoção)

Lista após remover o meio: 9

- 16 Crie um programa em C++ que delete o último nó de uma lista encadeada simples. O programa deve criar uma lista, exibi-la, remover o último elemento e exibir a lista atualizada. O processo pode ser repetido múltiplas vezes.

Exemplo de entrada

Lista original: 7 5 3 1

Exemplo de saída (após primeira remoção)

Lista após remoção do último nó: 7 5 3

Exemplo de saída (após segunda remoção)

Lista após remoção do último nó: 7 5

- 17 Crie um programa em C++ que implemente um array dinâmico simples com redimensionamento automático. O programa deve criar uma classe que armazena elementos em um array, e quando o array está cheio, cria um novo array com o dobro do tamanho, copia os elementos e libera a memória antiga.

Pseudo-código

```
insert(valor):  
    se tamanho = capacidade:  
        novo_array = novo array[capacidade * 2]  
        copiar elementos para novo_array  
        liberar array antigo  
        array = novo_array  
        capacidade = capacidade * 2  
    array[tamanho] = valor  
    tamanho = tamanho + 1
```

Exemplo de entrada

```
Digite os elementos (0 para parar):  
Elemento 1: 10  
Elemento 2: 20  
Elemento 3: 30  
Elemento 4: 40  
Elemento 5: 50  
Elemento 6: 0
```

Exemplo de saída

```
Capacidade inicial: 4  
Redimensionando: capacidade 4 -> 8  
Array final: 10 20 30 40 50  
Tamanho: 5  
Capacidade: 8
```

- 18 Crie um programa em C++ que implemente uma classe Vector com comportamento similar à lista do Python. A classe deve suportar: adicionar elemento no final (append), remover e retornar último elemento (pop), inserir em posição específica (insert), remover primeira ocorrência de valor (remove), acesso por índice (operator[]), e redimensionamento automático (dobrar quando cheio, reduzir pela metade quando muito vazio).

Pseudo-código

```
append(valor):
    se tamanho = capacidade: redimensionar()
    array[tamanho] = valor
    tamanho = tamanho + 1
```

```
insert(pos, valor):
    se tamanho = capacidade: redimensionar()
    para i de tamanho até pos + 1:
        array[i] = array[i-1]
    array[pos] = valor
    tamanho = tamanho + 1
```

```
pop():
    tamanho = tamanho - 1
    retornar array[tamanho]
```

```
remove(valor):
    para i de 0 até tamanho-1:
        se array[i] = valor:
            para j de i até tamanho-1:
                array[j] = array[j+1]
            tamanho = tamanho - 1
    retornar
```

Exemplo de interação

```
Vector v;
v.append(10);
v.append(20);
v.append(30);
v.display();           // [10, 20, 30]
v.insert(1, 15);       // [10, 15, 20, 30]
v.remove(20);          // [10, 15, 30]
int x = v.pop();       // x = 30, v = [10, 15]
v.display();           // [10, 15]
v[0] = 100;            // [100, 15]
```

Exemplo de saída

```
Tamanho: 2
Capacidade: 4
```

- 19 Crie um programa em C++ que implemente um sistema de Undo/Redo usando listas duplamente ligadas. O sistema deve permitir executar ações (com descrição), desfazer a última ação (undo) e refazer ações desfeitas (redo). Use duas listas: uma para ações executadas e outra para ações desfeitas.

Exemplo de interação

```
> add 10
Ação: Adicionar 10
> add 20
Ação: Adicionar 20
> multiply 2
Ação: Multiplicar por 2
> undo
Desfeito: Multiplicar por 2
> undo
Desfeito: Adicionar 20
> redo
Refeito: Adicionar 20
> display
Ações ativas: Adicionar 10, Adicionar 20
```

Exemplo de saída

```
Histórico de ações:
[Adicionar 10] <- [Adicionar 20] <- atual
Ações desfeitas: [Multiplicar por 2]
```

FILAS

- 20 Crie um programa em C++ que implemente a operação de enqueue (enfileirar) em uma fila usando array. O programa deve permitir ao usuário inserir elementos na fila até atingir a capacidade máxima, exibindo mensagens apropriadas.

Exemplo de entrada

```
Capacidade da fila: 5
Digite os elementos (0 para parar):
Elemento: 10
Elemento: 20
Elemento: 30
Elemento: 0
```

Exemplo de saída

```
Fila: [10, 20, 30]
Tamanho: 3
Capacidade: 5
```

- 21 Crie um programa em C++ que implemente a operação de dequeue (desenfileirar) em uma fila usando array. O programa deve permitir remover elementos do início da fila e exibir o elemento removido. Trate o caso de fila vazia.

Exemplo de entrada

```
Fila inicial: [10, 20, 30, 40, 50]
Quantos elementos remover? 3
```

Exemplo de saída

```
Removido: 10
Removido: 20
Removido: 30
Fila atual: [40, 50]
```

- 22 Crie um programa em C++ que implemente a operação de front (consultar início) em uma fila. O programa deve permitir visualizar o elemento do início da fila sem removê-lo, útil para verificar qual elemento será processado a seguir.

Exemplo de entrada

Fila: [15, 25, 35, 45]

Exemplo de saída

Elemento do início: 15

Próximo a ser atendido: 15

Fila permanece: [15, 25, 35, 45]

- 23 Crie um programa em C++ que implemente as operações de size (tamanho) e isEmpty (verificar se vazia) em uma fila. O programa deve fornecer métodos para consultar quantos elementos estão na fila e verificar se ela está vazia antes de operações de remoção.

Exemplo de interação

Fila vazia? Sim

Inserindo: 10, 20, 30

Tamanho: 3

Fila vazia? Não

Removendo todos...

Fila vazia? Sim

Tentando remover de fila vazia: ERRO - Fila vazia!

- 24 Crie um programa em C++ que implemente uma fila completa usando lista encadeada. Ao contrário da implementação com array, a fila deve crescer dinamicamente conforme necessário, sem limite pré-definido de capacidade.

Pseudo-código

```
enqueue(valor):  
    novo = criar_novo_no(valor)  
    se front = null:  
        front = novo  
        rear = novo  
    senão:  
        rear.next = novo  
        rear = novo  
  
dequeue():  
    valor = front.valor  
    front = front.next  
    se front = null:  
        rear = null  
    retornar valor
```

Exemplo de entrada

```
Digite elementos (0 para parar):  
Elemento: 5  
Elemento: 10  
Elemento: 15  
Elemento: 20  
Elemento: 0
```

Exemplo de saída

```
Fila: 5 -> 10 -> 15 -> 20  
Tamanho: 4  
Desenfileirando: 5  
Fila: 10 -> 15 -> 20
```


- 25 Crie um programa em C++ que simule um sistema de atendimento bancário com múltiplos caixas. Clientes chegam e são atendidos em ordem de chegada (FIFO) pelos caixas disponíveis. O sistema deve calcular métricas como tempo médio de espera na fila e tempo médio de atendimento por caixa.

Cada cliente tem um tempo de atendimento aleatório ou pré-definido. O sistema processa clientes até que todos sejam atendidos, distribuindo-os entre os caixas disponíveis.

Exemplo de entrada

```
Número de caixas: 3
Número de clientes: 8
Tempo de atendimento do cliente 1: 5 min
Tempo de atendimento do cliente 2: 3 min
Tempo de atendimento do cliente 3: 8 min
Tempo de atendimento do cliente 4: 2 min
Tempo de atendimento do cliente 5: 6 min
Tempo de atendimento do cliente 6: 4 min
Tempo de atendimento do cliente 7: 7 min
Tempo de atendimento do cliente 8: 3 min
```

Exemplo de saída

```
Distribuição de atendimento:
Caixa 1: Clientes 1(5min), 4(2min), 7(7min) - Total: 14min
Caixa 2: Clientes 2(3min), 5(6min), 8(3min) - Total: 12min
Caixa 3: Clientes 3(8min), 6(4min) - Total: 12min
```

Estatísticas:

```
Tempo médio de espera na fila: 4.25 min
Tempo médio de atendimento: 4.75 min
Caixa mais ocupado: Caixa 1 (14 min)
Total de clientes atendidos: 8
```

- 26 Crie um programa em C++ que implemente um buffer circular (ring buffer) para comunicação entre processos produtor e consumidor. O buffer tem capacidade fixa e opera com política FIFO circular: quando cheio, o produtor espera; quando vazio, o consumidor espera.

O programa deve simular a produção e consumo de dados, mostrando o estado do buffer a cada operação e como os índices de inserção e remoção circulam pelo array.

Pseudo-código

```
enqueue(valor):
    se (in + 1) % capacidade = out:
        buffer cheio - esperar
    buffer[in] = valor
    in = (in + 1) % capacidade

dequeue():
    se in = out:
        buffer vazio - esperar
    valor = buffer[out]
    out = (out + 1) % capacidade
    retornar valor
```

Exemplo de simulação

Capacidade do buffer: 4

Produzir: A

Buffer: [A, _, _, _] (in=1, out=0)

Produzir: B

Buffer: [A, B, _, _] (in=2, out=0)

Consumir: A

Buffer: [_, B, _, _] (in=2, out=1)

Produzir: C

Buffer: [_, B, C, _] (in=3, out=1)

Produzir: D

Buffer: [_, B, C, D] (in=0, out=1) // circular!

Produzir: E

Buffer: [E, B, C, D] (in=1, out=1) // sobrescreve? Não, espera!

Exemplo de saída

Consumir: B
Buffer: [E, C, D, _] (in=1, out=2)
Total produzido: 5 itens

Total consumido: 2 itens

- 27 Crie um programa em C++ que implemente uma fila de impressão circular com limite de documentos. O sistema gerencia documentos enviados por múltiplos usuários para uma impressora compartilhada. Quando a fila atinge a capacidade máxima, novos documentos substituem os mais antigos (política de substituição circular).

Cada documento tem: ID, nome do usuário, nome do arquivo e número de páginas. O sistema deve mostrar a ordem de impressão e quais documentos foram substituídos quando a fila estava cheia.

Exemplo de entrada

Capacidade da fila: 5 documentos

Usuário Ana envia: "relatorio.pdf" (10 páginas)
Usuário Bruno envia: "foto.jpg" (1 página)
Usuário Carla envia: "contrato.doc" (5 páginas)
Usuário Daniel envia: "apresentacao.ppt" (15 páginas)
Usuário Elisa envia: "planilha.xlsx" (3 páginas)

Fila cheia! (5/5 documentos)

Usuário Fabio envia: "artigo.pdf" (8 páginas)

Exemplo de saída

Fila de impressão:

1. Bruno - foto.jpg (1 pág)
2. Carla - contrato.doc (5 pág)
3. Daniel - apresentacao.ppt (15 pág)
4. Elisa - planilha.xlsx (3 pág)
5. Fabio - artigo.pdf (8 pág)

Documento substituído: Ana - relatorio.pdf (10 pág)

Total de páginas na fila: 32

Tempo estimado: 6 min 24 seg

ORDENAÇÃO

- 28 Crie um programa em C++ que implemente o Bubble Sort para ordenar um array de inteiros em ordem crescente.

Pseudo-código

```
para i de 0 até n-2:
    para j de 0 até n-i-2:
        se A[j] > A[j+1]:
            trocar(A[j], A[j+1])
```

Exemplo de entrada

```
Digite o tamanho do array: 6
Digite os elementos:
Elemento 1: 64
Elemento 2: 34
Elemento 3: 25
Elemento 4: 12
Elemento 5: 22
Elemento 6: 11
```

Exemplo de saída

```
Array original: 64 34 25 12 22 11
Array ordenado: 11 12 22 25 34 64
Total de comparações: 15
Total de trocas: 9
```

- 29 Crie um programa em C++ que implemente o Selection Sort para ordenar um array de inteiros.

Pseudo-código

```
para i de 0 até n-1:
    min = i
    para j de i+1 até n-1:
        se A[j] < A[min]:
            min = j
    se min ≠ i:
        trocar(A[i], A[min])
```

Exemplo de entrada

Digite o tamanho do array: 5
Digite os elementos:
Elemento 1: 29
Elemento 2: 10
Elemento 3: 14
Elemento 4: 37
Elemento 5: 13

Exemplo de saída

Array original: 29 10 14 37 13
Passo 1: Encontrou menor 10, trocou com posição 0: 10 29 14 37 13
Passo 2: Encontrou menor 13, trocou com posição 1: 10 13 14 37 29
Passo 3: Encontrou menor 14, trocou com posição 2: 10 13 14 37 29
Passo 4: Encontrou menor 29, trocou com posição 3: 10 13 14 29 37
Array ordenado: 10 13 14 29 37

- 30 Crie um programa em C++ que implemente o Insertion Sort para ordenar um array de inteiros.

Pseudo-código

```
para i de 1 até n-1:
    chave = A[i]
    j = i - 1
    enquanto j ≥ 0 e A[j] > chave:
        A[j+1] = A[j]
        j = j - 1
    A[j+1] = chave
```

Exemplo de entrada

```
Digite o tamanho do array: 6
Digite os elementos:
Elemento 1: 12
Elemento 2: 11
Elemento 3: 13
Elemento 4: 5
Elemento 5: 6
Elemento 6: 2
```

Exemplo de saída

```
Array original: 12 11 13 5 6 2
Inserindo 11: 11 12 13 5 6 2
Inserindo 13: 11 12 13 5 6 2
Inserindo 5: 5 11 12 13 6 2
Inserindo 6: 5 6 11 12 13 2
Inserindo 2: 2 5 6 11 12 13
Array ordenado: 2 5 6 11 12 13
```

- 31 Crie um programa em C++ que implemente o Merge Sort.

Pseudo-código

```
mergeSort(A, esq, dir):  
    se esq < dir:  
        meio = (esq + dir) / 2  
        mergeSort(A, esq, meio)  
        mergeSort(A, meio+1, dir)  
        merge(A, esq, meio, dir)  
  
merge(A, esq, meio, dir):  
    // Combinar dois subarrays ordenados
```

Exemplo de entrada

```
Digite o tamanho do array: 8  
Digite os elementos:  
Elemento 1: 38  
Elemento 2: 27  
Elemento 3: 43  
Elemento 4: 3  
Elemento 5: 9  
Elemento 6: 82  
Elemento 7: 10  
Elemento 8: 50
```

Exemplo de saída

```
Array original: 38 27 43 3 9 82 10 50  
Dividindo: [38 27 43 3] [9 82 10 50]  
Dividindo: [38 27] [43 3]  
Dividindo: [38] [27]  
Mesclando [27 38]  
Dividindo: [43] [3]  
Mesclando [3 43]  
Mesclando [3 27 38 43]  
Dividindo: [9 82] [10 50]  
Dividindo: [9] [82]  
Mesclando [9 82]  
Dividindo: [10] [50]  
Mesclando [10 50]  
Mesclando [9 10 50 82]  
Mesclando [3 9 10 27 38 43 50 82]  
Array ordenado: 3 9 10 27 38 43 50 82
```

- 32 Crie um programa em C++ que implemente o Quick Sort.

Pseudo-código

```
quickSort(A, esq, dir):  
    se esq < dir:  
        p = particionar(A, esq, dir)  
        quickSort(A, esq, p-1)  
        quickSort(A, p+1, dir)
```

```
particionar(A, esq, dir):  
    pivô = A[(esq + dir) / 2]  
    // Particionar ao redor do pivô
```

Exemplo de entrada

```
Digite o tamanho do array: 7  
Digite os elementos:  
Elemento 1: 25  
Elemento 2: 13  
Elemento 3: 6  
Elemento 4: 42  
Elemento 5: 17  
Elemento 6: 8  
Elemento 7: 30
```

Exemplo de saída

```
Array original: 25 13 6 42 17 8 30  
Particionando [25 13 6 42 17 8 30], pivô: 17  
Esquerda: [13 6 8] | Direita: [25 42 30]  
Particionando [13 6 8], pivô: 6  
Esquerda: [] | Direita: [13 8]  
Particionando [13 8], pivô: 8  
Esquerda: [] | Direita: [13]  
Ordenado: [6 8 13]  
Particionando [25 42 30], pivô: 42  
Esquerda: [25 30] | Direita: []  
Particionando [25 30], pivô: 30  
Esquerda: [25] | Direita: []  
Ordenado: [25 30 42]  
Array ordenado: 6 8 13 17 25 30 42
```


EXERCÍCIOS DE ENADE

Ano	#	Curso	Conteúdo
2011	20	Ciência da Computação	Teoria de Grafos
2011	21	Ciência da Computação	Pilha e Fila no contexto da Genética
2011	24	Ciência da Computação	Fila de Prioridade (Heap)
2011	28	Ciência da Computação	Análise e Projeto de Algoritmos
2011	30	Ciência da Computação	Árvore Binária
2011	D03	Ciência da Computação	Fibonacci Recursivo
2011	D04	Ciência da Computação	Algoritmos de Ordenação e Listas Ordenadas
2014	D03	Ciência da Computação	Busca Recursiva
2014	D05	Ciência da Computação	Análise e Projeto de Algoritmos
2014	12	Ciência da Computação	Análise de Algoritmos
2014	13	Ciência da Computação	Árvore Binária
2014	16	Ciência da Computação	Pilha em C
2014	23	Ciência da Computação	Algoritmos Recursivos
2017	D03	Ciência da Computação	Lista Ligada
2017	D05	Ciência da Computação	Busca em Profundidade
2017	09	Ciência da Computação	Árvore Balanceada (AVL)
2017	18	Ciência da Computação	Algoritmos de Ordenação, Array em C
2017	22	Ciência da Computação	Algoritmo Guloso
2017	24	Ciência da Computação	Algoritmos em Grafos

Ano	#	Curso	Conteúdo
2017	25	Ciência da Computação	Análise de Fibonacci Recursivo
2017	33	Ciência da Computação	Função Recursiva
2019	D03	Engenharia da Computação	Árvore Binária e AVL
2019	D04	Engenharia da Computação	Produtório Iterativo vs Recursivo
2019	09	Engenharia da Computação	Algoritmo de Ordenação
2019	24	Engenharia da Computação	Spanning Tree no contexto de Algoritmos de Roteamento
2019	25	Engenharia da Computação	Busca Recursiva em Python
2021	D05	Ciência da Computação	Heap Binário
2021	20	Ciência da Computação	Análise de Algoritmos em C
2021	23	Ciência da Computação	Árvore Binária
2021	32	Ciência da Computação	Algoritmos de Ordenação
2021	34	Ciência da Computação	Algoritmo de Dijkstra
2023	D02	Engenharia da Computação	Algoritmo de Ordenação
2023	15	Engenharia da Computação	Vazamento de Memória
2023	18	Engenharia da Computação	Arrays Dinâmicos
2023	32	Engenharia da Computação	Busca Gulosa em Grafos
2023	34	Engenharia da Computação	Filas no contexto de Redes de Computadores

EXERCÍCIOS SABOR ENADE